

EF CORE 8 / 9 ARCHITECTURE

Relaciones en EF Core Moderno

Guía Basada en Convenciones Automáticas, Fluent API en ApplicationDbContext, Carga de Datos y Operaciones C.R.U.D. (Sin Anotaciones de Datos)

Convenciones y Ubicación de Configuración

Cómo Entity Framework Core deduce relaciones automáticamente y de qué forma
organizar Fluent API en tu proyecto

I Filosofía Moderna: Cero Anotaciones

Convenciones Inteligentes

EF Core detecta Claves Primarias (`Id`, `ClaseId`), Claves Foráneas y Relaciones simplemente reconociendo propiedades de navegación (objetos o colecciones).

No se requieren atributos ni Data Annotations (`[ForeignKey]`, `[Key]`) en las clases de modelo.

Modelos POCO Limpios

Tus entidades de dominio permanecen libres de dependencias de persistencia y metadatos visuales.

La configuración explícita se reserva **únicamente para Fluent API** cuando existen reglas no estándar.

I ¿Dónde se configura Fluent API?

Opción A: Directo en ApplicationDbContext

Mediante la sobrescritura del método `OnModelCreating(ModelBuilder modelBuilder)` dentro de tu clase `ApplicationDbContext` (que hereda de `DbContext`).

Ideal para proyectos pequeños o medianos donde se prefiere tener la configuración centralizada.

Opción B: Clases `IEntityTypeConfiguration`

Creando clases independientes por entidad que implementan `IEntityTypeConfiguration` con el método `Configure(EntityTypeBuilder builder)`.

Ideal para aplicaciones grandes donde se requiere desacoplamiento modular.

| Opción A: Fluent API en AppDbContext

Implementación Directa

```
public class AppDbContext : DbContext {  
    public DbSet<Blog> Blogs => Set<Blog>();  
    public DbSet<Post> Posts => Set<Post>();  
  
    protected override void OnModelCreating(ModelBuilder modelBuilder) {  
        base.OnModelCreating(modelBuilder);  
  
        // Fluent API se escribe directamente aquí:  
        modelBuilder.Entity<Post>()  
            .Property(p => p.Title).HasMaxLength(200);  
    }  
}
```

Ventajas de esta Opción

- **Simplicidad:** Toda la configuración de Fluent API reside en un solo archivo accesible.
- **Directa:** No requiere instanciar ni escanear clases secundarias.
- **Transición fácil:** Permite agregar Fluent API de forma paulatina sobre la marcha.

| Opción B: Separación con IEntityConfiguration

1. Definición de la Configuración

```
public class PostConfiguration : IEntityConfiguration<Post> {  
    public void Configure(EntityTypeBuilder<Post> builder) {  
        builder.Property(p => p.Title).HasMaxLength(200);  
  
        builder.HasOne(p => p.Blog)  
            .WithMany(b => b.Posts)  
            .OnDelete(DeleteBehavior.Restrict);  
    }  
}
```

2. Registro en AppDbContext

```
public class AppDbContext : DbContext {  
    protected override void OnModelCreating(ModelBuilder modelBuilder)  
    {  
        base.OnModelCreating(modelBuilder);  
  
        // Aplica todas las configuraciones del Assembly automáticamente  
        modelBuilder.ApplyConfigurationsFromAssembly(  
            typeof(AppDbContext).Assembly);  
    }  
}
```

Relaciones Uno a Muchos (1:N)

El patrón más común: un registro padre vinculado con múltiples registros hijos

1:N por Convención Pura (Cero Fluent API)

Modelos C# Simplificados

```
public class Blog {  
    public int Id { get; set; }  
    public string Url { get; set; } = "";  
    public List<Post> Posts { get; set; } = [];  
}  
  
public class Post {  
    public int Id { get; set; }  
    public string Title { get; set; } = "";  
    public int BlogId { get; set; } // FK detectada  
    public Blog Blog { get; set; } = null!; // Navegación  
}
```

Detección Automática de EF Core

- Detecta la propiedad `BlogId` como Clave Foránea apuntando a `Blog.Id`.
- Infiere la relación 1:N entre `Blog.Posts` y `Post.Blog`.

✓ ¡No requiere escribir nada en `OnModelCreating` ni atributos!

| 1:N: Nulabilidad de la Clave Foránea

Relación Requerida (`int BlogId`)

Propiedad FK de tipo entero no nulo.

- Un `Post` debe pertenecer obligatoriamente a un `Blog`.
- EF Core aplica borrado en cascada (`DeleteBehavior.Cascade`) automáticamente.
- La columna SQL se genera como `NOT NULL`.

Relación Opcional (`int? BlogId`)

Propiedad FK de tipo entero nulo.

- Un `Post` puede existir de forma independiente sin pertenecer a ningún `Blog`.
- EF Core asigna `ClientSetNull` al borrar el padre.
- La columna SQL se genera como `NULL`.

1:N: ¿Cuándo usar Fluent API?

Ajustar Regla de Borrado

Para evitar el borrado en cascada predeterminado en relaciones requeridas:

```
// En AppDbContext.OnModelCreating:  
modelBuilder.Entity<Post>()  
    .HasOne(p => p.Blog)  
    .WithMany(b => b.Posts)  
    .OnDelete(DeleteBehavior.Restrict);
```

Nombre No Estándar de FK

Si la clave foránea no sigue el patrón NombreEntidadId:

```
public int ParentBlogRef { get; set; }  
  
// En AppDbContext.OnModelCreating:  
modelBuilder.Entity<Post>()  
    .HasOne(p => p.Blog)  
    .WithMany(b => b.Posts)  
    .HasForeignKey(p => p.ParentBlogRef);
```

Relaciones Uno a Uno (1:1)

Modelado de correspondencias unívocas entre dos entidades sin configuraciones redundantes



1:1 por Convención Pura

Modelos C# Simplificados

```
public class User {
    public int Id { get; set; }
    public string Username { get; set; } = "";
    public UserProfile? Profile { get; set; }
}

public class UserProfile {
    public int Id { get; set; }
    public string Bio { get; set; } = "";
    public int UserId { get; set; } // FK explícita
    public User User { get; set; } = null!;
}
```

Cómo opera EF Core

- Detecta la relación 1:1 al ver navegaciones de referencia única en ambas clases.
- Al encontrar `UserId` en `UserProfile`, entiende que `UserProfile` es la entidad **dependiente**.
- Crea automáticamente un **Índice Único (Unique Index)** sobre la FK en la BD.

1:1: Entidad Principal vs Dependiente

¿Quién lleva la Clave Foránea?

La entidad que incluye la propiedad FK (`UserId`) es la entidad dependiente.

Al eliminar el `User` (Principal), el `UserProfile` (Dependiente) se eliminará automáticamente en cascada.

¿Cuándo usar Fluent API en 1:1?

Solo si **ninguna de las clases declara la propiedad de clave foránea** y EF Core no puede inferir cuál es el dependiente:

```
modelBuilder.Entity<User>()  
    .HasOne(u => u.Profile)  
    .WithOne(p => p.User)  
    .HasForeignKey<UserProfile>(p => p.UserId);
```

Relaciones Muchos a Muchos (N:M)

Mapeos bidireccionales administrados en forma directa o mediante tablas con campos de datos extra

| N:M Implícita (EF Core 5+)

Modelos C# Simplificados

```
public class Post {  
    public int Id { get; set; }  
    public string Title { get; set; } = "";  
    public List<Tag> Tags { get; set; } = [];  
}  
  
public class Tag {  
    public int Id { get; set; }  
    public string Name { get; set; } = "";  
    public List<Post> Posts { get; set; } = [];  
}
```

Cero Tablas Intermedias en C#

- EF Core crea y administra automáticamente en la base de datos la tabla de unión Post Tag.
- No requiere declarar una clase intermedia ni configurar nada en Fluent API.

✓ Agrega o remueve elementos directamente en las listas C#.

I N:M Explícita (Con Payload / Datos Extra)

Entidad Intermedia Personalizada

Requerida cuando la tabla de unión posee información adicional (ej. AssignedAt):

```
public class PostTag {  
    public int PostId { get; set; }  
    public Post Post { get; set; } = null!;  
  
    public int TagId { get; set; }  
    public Tag Tag { get; set; } = null!;  
  
    public DateTime AssignedAt { get; set; }  
}
```

Ajuste en Entidades Principales

Las colecciones principales apuntan ahora a la entidad intermedia:

```
public class Post {  
    public int Id { get; set; }  
    public List<PostTag> PostTags { get; set; } = [];  
}  
  
public class Tag {  
    public int Id { get; set; }  
    public List<PostTag> PostTags { get; set; } = [];  
}
```

I N:M Explícita: Configuración en AppDbContext

Configurar Clave Compuesta

La tabla intermedia requiere una clave primaria compuesta definida con Fluent API:

```
protected override void OnModelCreating(ModelBuilder modelBuilder) {  
    base.OnModelCreating(modelBuilder);  
  
    // Define la PK compuesta de la tabla de unión:  
    modelBuilder.Entity<PostTag>()  
        .HasKey(pt => new { pt.PostId, pt.TagId });  
}
```

¿Por qué no hace falta definir más?

- Las dos relaciones 1:N individuales (Post -> PostTag y Tag -> PostTag) son detectadas por convención gracias a las propiedades PostId y TagId.
- ¡No necesitas encadenar HasOne / WithMany redundantes!

Integridad Referencial y Mapeos Especiales

Comportamientos de borrado y propiedades en sombra administradas internamente

| Comportamientos de Borrado (DeleteBehavior)

DeleteBehavior	Efecto en Base de Datos / EF Core	Caso de Uso Recomendado
Cascade	Elimina automáticamente todos los registros dependientes.	Composición fuerte (ej. Pedido -> DetallePedido). Valor por defecto en FKs no nulas.
SetNull	Establece la clave foránea en NULL en los registros dependientes.	Relaciones opcionales (int?) donde el hijo puede sobrevivir sin padre.
Restrict	Impide la eliminación del registro padre si tiene hijos asociados.	Protección estricta de catálogos maestros (ej. Categoria -> Productos).
ClientSetNull	EF Core limpia la relación en memoria; la BD fallará si es NOT NULL.	Comportamiento por defecto para relaciones opcionales.

| Propiedades en Sombra (Shadow Properties)

¿Qué son?

Columnas que existen en el modelo de la base de datos pero **no están declaradas como propiedades en la clase C#**.

Si defines la navegación `public Blog Blog { get; set; }` sin declarar la propiedad `int BlogId`, EF Core genera una Shadow Property llamada `BlogId`.

Filtrar en LINQ con EF.Property

Aunque la propiedad no exista explícitamente en C#, puedes consultarla en LINQ:

```
var posts = await context.Posts
    .Where(p => EF.Property<int>(p, "BlogId") = 5)
    .ToListAsync();
```

Carga de Datos Relacionados

Estrategias avanzadas en consultas LINQ para recuperar grafos de objetos

| Carga Anticipada (Eager Loading)



Include / ThenInclude

Trae entidades relacionadas en la misma consulta mediante JOINS:

```
context.Blogs
    .Include(b => b.Posts)
    .ThenInclude(p => p.Author)
```



Filtered Include

Aplica filtros LINQ a las colecciones incluidas:

```
context.Blogs
    .Include(b => b.Posts
        .Where(p => p.IsActive))
```



Auto-Include

Configura una navegación para cargarse siempre:

```
modelBuilder.Entity<Blog>()
    .Navigation(b => b.Posts)
    .AutoInclude();
```

| Carga Explícita y Diferida (Lazy)

Carga Explícita (Explicit)

Carga relaciones bajo demanda para entidades ya rastreadas en memoria:

```
var blog = await context.Blogs.FirstAsync();  
  
// Cargar colección explícitamente:  
await context.Entry(blog)  
    .Collection(b => b.Posts)  
    .LoadAsync();
```

Carga Diferida (Lazy Loading)

Carga automática al acceder a propiedades de navegación declaradas como `virtual`.

- Requiere `UseLazyLoadingProxies()`.

⚠ Atención: Puede provocar el problema N+1 si se usa dentro de bucles.

Proyecciones LINQ Eficientes (Select)

Consulta con Proyección a DTO

```
var result = await context.Blogs
    .Select(b => new BlogDto {
        Id = b.Id,
        Url = b.Url,
        TotalPosts = b.Posts.Count(),
        Titles = b.Posts
            .Select(p => p.Title)
            .ToList()
    })
    .ToListAsync();
```

Ventajas de Rendimiento

- **Optimización SQL:** EF Core genera un SELECT trayendo únicamente las columnas requeridas.
- **Sin Include:** No necesitas llamar a `Include()`.
- **Sin Change Tracker:** Los DTOs no se rastrean, reduciendo el consumo de RAM.

| Consultas Divididas (AsSplitQuery)

El Problema Cartesiano

Al incluir múltiples colecciones con `Include`, SQL genera un producto cartesiano que duplica filas masivamente, ralentizando la red y saturando la memoria.

Solución Split Query

Ejecuta sentencias SQL independientes para cada relación y las combina de forma eficiente en C#:

```
var blogs = await context.Blogs
    .Include(b => b.Posts)
    .Include(b => b.Contributors)
    .AsSplitQuery()
    .ToListAsync();
```

Operaciones C.R.U.D. en Grafos

Persistencia, vinculación y eliminación de entidades relacionadas

I Inserción y Modificación de Grafos

Inserción en Cascadas

Al guardar un objeto principal, EF Core asigna las FKs e inserta automáticamente todo su grafo de objetos asociados:

```
var blog = new Blog {  
    Url = "https://dev.to",  
    Posts = [  
        new Post { Title = "EF Core 9 News" }  
    ]  
};  
  
context.Blogs.Add(blog);  
await context.SaveChangesAsync();
```

Actualización de Vinculación

Para cambiar el padre de un objeto, basta con reasignar su propiedad FK o navegación:

```
post.BlogId = 2; // Cambia de Blog  
await context.SaveChangesAsync();
```

| Gestión de Huérfanos (Orphan Removal)

Eliminación Automática en 1:N Requeridas

En relaciones no nulas (NOT NULL), eliminar un objeto de la colección del padre hace que quede huérfano:

```
blog.Posts.Remove(post);  
  
// EF Core detecta que quedó sin padre  
// y emite un DELETE en SaveChanges()  
await context.SaveChangesAsync();
```

Comportamiento en Relaciones Opcionales

Si la FK es opcional (`int?`) y se remueve el elemento de la lista, EF Core no borrará el registro de la BD, sino que actualizará su clave foránea a NULL.

¿Preguntas?

Aprovecha las convenciones automáticas de EF Core y usa Fluent API en ApplicationDbContext solo cuando sea necesario.

Estructura limpia + Convenciones automáticas + Proyecciones Select = Código .NET Eficiente